چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۴

معماری لایهٔ داده — بخش ۲: offline-first، sync و مقیاس



اسماعیل مرادی

@esmaeeil-moradi700

چرا offline-first روز چهاردهم است؟

روز ۱۳ repositoryها، مدل‌ها و cacheها را طراحی کرد. بخش ۲ جایی است که طراحی‌ها در production می‌شکنند: **نوشتن بدون شبکه**، دو دستگاه که یک رکورد را ویرایش می‌کنند، و تغییر اسکیما روی دستگاه‌هایی که نمی‌توانید به عقب برگردانید.

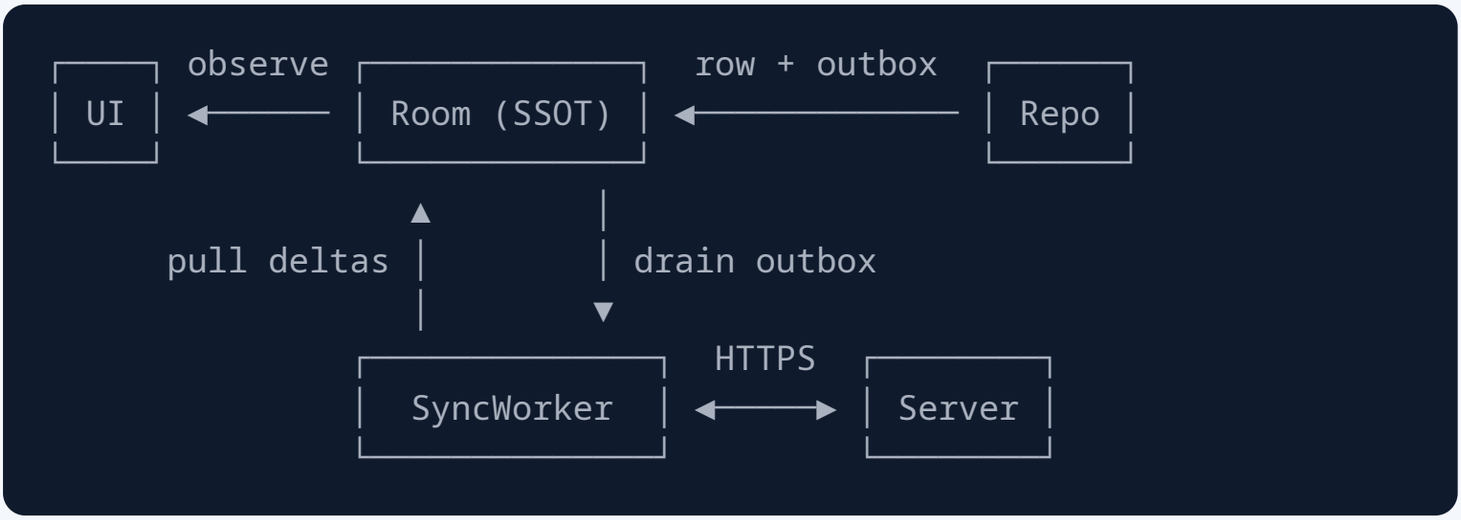
- خواندن آفلاین آسان است — **نوشتن آفلاین** و **sync** بخش سخت ماجراست
- تعارض‌ها: تشخیص‌شان و حل‌شان برای هر نوع داده
- صفحه‌بندی لیست‌های بزرگ از شبکه **و** دیتابیس با یک منبع حقیقت
- migration و رمزنگاری داده‌ای که باید سال‌ها دوام بیاورد
- یک لایهٔ داده برای اندروید و iOS با Kotlin Multiplatform

سطح	سؤال‌ها
جونیور	معنای offline-first، نوشتن آفلاین، چرا WorkManager برای sync
مید	الگوی outbox، worker مربوط به sync، دریافت تغییرات، صفحه‌بندی با RemoteMediator
سینیور	تشخیص تعارض، حل تعارض، migration در مقیاس بزرگ
آرکیتکت	رمزنگاری داده روی دیسک، لایهٔ داده در KMP، sync برای پول



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: اول محلی، sync در پس‌زمینه



- **UI فقط از دیتابیس محلی می‌خواند** — آنلاین یا آفلاین، یک مسیر کد
- **نوشتن‌ها اول محلی ثبت می‌شوند**، همراه با یک ورودی outbox، در یک تراکنش
- **یک worker پس‌زمینه sync می‌کند:** outbox را می‌فرستد، تغییرات سرور را می‌گیرد و تعارض‌ها را حل می‌کند

دو حقیقت، در دو سطح

روی دستگاه، دیتابیس منبع واحد حقیقت **برای UI** است. بین دستگاه‌ها، **سرور** مرجعی است که تصمیم می‌گیرد نوشتن‌های متعارض چطور تمام شوند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

معنای offline-first، نوشتن آفلاین، چرا WorkManager برای sync



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. offline-first واقعاً یعنی چه؟

تعریف اندروید: اپی که می‌تواند **همه یا بخش حیاتی** از کارکرد اصلی‌اش را بدون اینترنت انجام دهد.

	اپ آنلاین با cache	اپ offline-first
خواندن	از شبکه؛ cache به‌عنوان پشتیبان	همیشه از دیتابیس محلی
نوشتن	بدون شبکه شکست می‌خورد	محلی ذخیره و بعداً sync می‌شود
شبکه	روی مسیر حیاتی	در پس‌زمینه
حالت بارگذاری	یک spinner برای هر درخواست	اول محتوا، وضعیت sync در کنار

تصمیم بگیرید **کدام** فیچرها باید آفلاین کار کنند — offline-first یک تصمیم محصولی برای هر فیچر است (روز ۱۳، سؤال ۱۲).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*offline-first یعنی cache کردن پاسخ‌های API.*» — «cache به خواندن کمک می‌کند. offline-first یعنی **پذیرفتن نوشتن** بدون شبکه و آشتی دادنش در آینده — کار اصلی طراحی همین‌جاست.

تلهٔ مصاحبه

«*قبل از هر درخواست اتصال را چک کن.*» — «شبکه در دسترس است» یعنی سرور شما در دسترس است؟ نه لزوماً (captive portal، VPN، قطعی سرور). درخواست را بفرستید، شکست را مدیریت کنید، و بگذارید شرط شبکهٔ WorkManager زمان تلاش دوباره را تعیین کند. callbackهای اتصال فقط برای راهنمایی در UX.

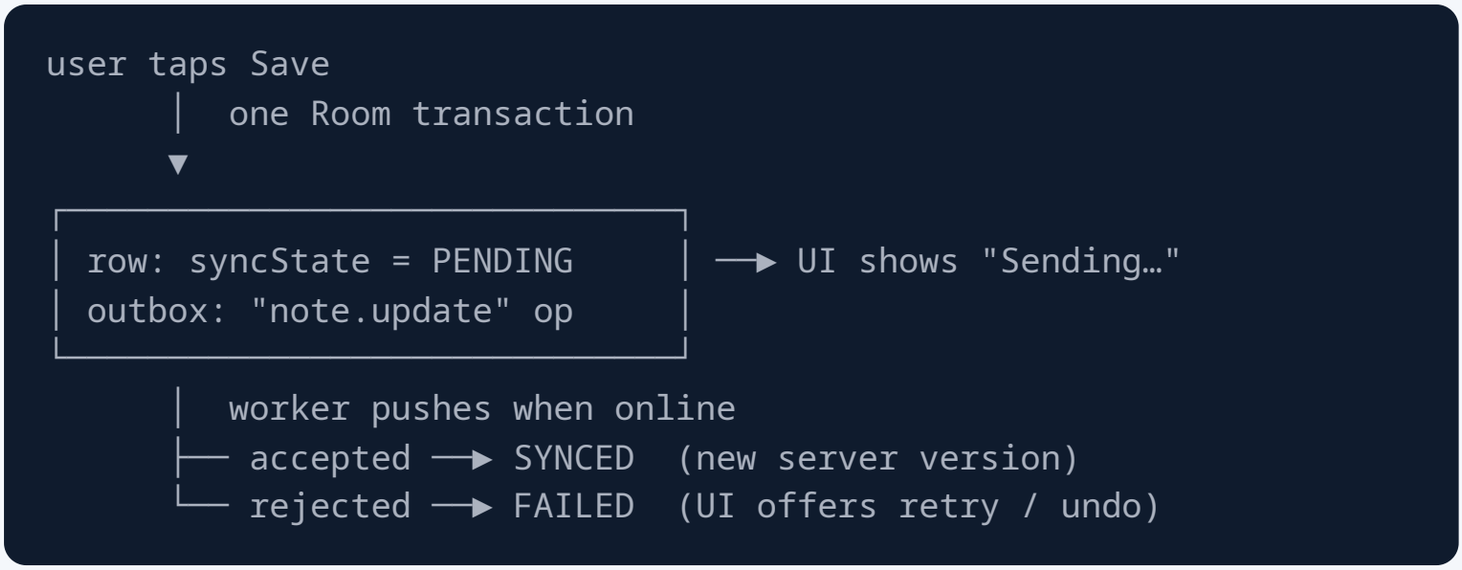
سؤال تکمیلی

«*اگر سرور مرجع است، چطور دیتابیس محلی منبع حقیقت است؟*» — سطح‌ها فرق دارند: UI **فقط** دیتابیس را observe می‌کند، پس صفحه همیشه سازگار است؛ سرور **تعارض‌ها** را حل می‌کند و نتیجه‌اش دوباره در دیتابیس نوشته می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. زندگی یک نوشتن آفلاین



- کاربر تغییرش را **فوراً** می‌بیند — چون همین حالا در دیتابیس است
- وضعیت sync خودش **داده** است: در ردیف ذخیره‌اش کنید و مثل هر فیلد دیگری رندرش کنید
- نوشتنی که رد شده باید دیده شود؛ برگرداندن بی‌صدای آن از نگاه کاربر یعنی از دست رفتن داده



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. جاهایی که نوشتن خوش‌بینانه خراب می‌شود

تلهٔ مصاحبه

«*UI خوش‌بینانه یعنی state در ViewModel را عوض کن و API را صدا بزن.*» — process death یا یک درخواست ناموفق تغییر را گم می‌کند. اول ذخیره، بعد sync: ویرایشِ در انتظار را دیتابیس نگه می‌دارد، نه ViewModel.

تلهٔ مصاحبه

«*وقتی آفلاین است دکمه را غیرفعال کن.*» — این همان اپ آنلاین است با چند قدم اضافه. اگر فیچر در فهرست آفلاین شماست، نوشتن را بپذیرید و وضعیت sync آن را نشان دهید.

سؤال تکمیلی

«*رکوردهایی که آفلاین ساخته می‌شوند چه شناسه‌ای می‌گیرند؟*» — شناسه‌ای که خود کلاینت می‌سازد (UUID)، تا ردیف‌های مرتبط و تلاش‌های دوباره قبل از اینکه سرور رکورد را ببیند به آن ارجاع دهند. دیگر شناسهٔ موقتی در کار نیست که بعداً بازنویسی شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. چرا sync را WorkManager اجرا می‌کند

```
class SyncScheduler(private val workManager: WorkManager) {
    private val online = Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .build()

    fun requestSync() {
        val request = OneTimeWorkRequestBuilder<SyncWorker>()
            .setConstraints(online)
            .build()
        workManager.enqueueUniqueWork(
            "sync",
            ExistingWorkPolicy.APPEND_OR_REPLACE,
            request,
        )
    }
}
```

- کار **ماندگار** است: از process death و ری‌استارت دستگاه عبور می‌کند
- وقتی **شرط** برقرار شد اجرا می‌شود؛ `Result.retry()` از backoff استفاده می‌کند (به‌طور پیش‌فرض نمایی)
- **کار یکتا (unique work):** ده نوشتن یعنی یک صف sync، نه ده sync موازی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. تله‌های WorkManager

تلهٔ مصاحبه

«*sync فقط روی شبکهٔ بدون محدودیت حجم*» (نسخهٔ پارسال این را ایده‌آل می‌دانست). — برای **نوشتن‌های کاربر** این یعنی «هرگز» برای کسی که فقط اینترنت موبایل دارد. برای تغییرات کاربر `CONNECTED`؛ و `UNMETERED` را برای پیش‌دریافت‌های حجیم نگه دارید.

تلهٔ مصاحبه

استفاده از `ExistingWorkPolicy.KEEP` برای syncی که با نوشتن فعال می‌شود. درخواستی که وقتی worker در حال تمام شدن است برسد دور ریخته می‌شود و تغییر جدید تا محرک بعدی منتظر می‌ماند. `APPEND_OR_REPLACE` یک اجرای دیگر بعد از اجرای فعلی در صف می‌گذارد.

تلهٔ مصاحبه

«*WorkManager اجرای فوری را تضمین می‌کند.*» — اجرای **نهایی** را وقتی شرط‌ها اجازه دهند تضمین می‌کند. سیستم می‌تواند عقبش بیندازد، و force-stop کارها را تا باز شدن دوبارهٔ اپ لغو می‌کند. کار expedited، حالت Doze و standby bucketها ← روز ۱۵.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

الگوی outbox، worker مربوط به sync، دریافت تغییرات، صفحه‌بندی با RemoteMediator



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. الگوی outbox

```
suspend fun addComment(postId: String, text: String) {
    val id = UUID.randomUUID().toString()
    val payload = json.encodeToString(
        NewCommentPayload(id, postId, text),
    )
    db.withTransaction {
        commentDao.upsert(
            CommentEntity(id, postId, text, SyncState.PENDING),
        )
        outboxDao.insert(
            OutboxEntry(opId = id, type = "comment.create",
                payload = payload),
        )
    }
    syncScheduler.requestSync()
}
```

ردیف و ورودی outbox **به‌صورت اتمیک** نوشته می‌شوند: هرگز کامنتِ در انتظاری بدون عملیاتِ در صف وجود ندارد، و برعکس.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. طراحی outbox

```
@Entity(tableName = "outbox")
data class OutboxEntry(
    @PrimaryKey(autoGenerate = true) val seq: Long = 0,
    val opId: String,          // idempotency key (header)
    val type: String,          // "comment.create", "note.update"
    val payload: String,       // versioned JSON
    val attempts: Int = 0,
)
```

- **idempotency key:** سرور آن را نگه می‌دارد؛ درخواست تکراری همان نتیجهٔ اول را برمی‌گرداند، نه یک رکورد تکراری
- **ترتیب:** صف را به ترتیب `seq` خالی کنید — ویرایش نباید قبل از ساختن به سرور برسد
- **ادغام** وقتی امن است: پنج ویرایش روی یک یادداشت می‌تواند یک update شود

تلهٔ مصاحبه

«تلاش دوباره امن است چون از WorkManager استفاده می‌کنیم.» — WorkManager خود **worker** را دوباره اجرا می‌کند؛ نمی‌داند سرور قبل از قطع اتصال درخواست را اعمال کرده یا نه. فقط idempotency key تلاش دوباره را امن می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. worker مربوط به sync

```
@HiltWorker
class SyncWorker @AssistedInject constructor(
    @Assisted ctx: Context,
    @Assisted params: WorkerParameters,
    private val outbox: OutboxDao,
    private val api: SyncApi,
) : CoroutineWorker(ctx, params) {
    override suspend fun doWork(): Result {
        while (true) {
            val op = outbox.oldest() ?: break
            when (api.push(op)) {
                PushResult.Accepted -> outbox.complete(op)
                PushResult.Rejected -> outbox.fail(op)
                PushResult.Transient -> return Result.retry()
            }
        }
        // then pull server changes (Q6)
        return Result.success()
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. قاعده‌های workerی که هرگز داده را خراب نمی‌کند

- تابع `complete(op)` ردیف را `SYNCED` می‌کند **و** ورودی را حذف می‌کند، در یک تراکنش
- تابع `push` خطاها را تبدیل می‌کند (همان `safeCall` روز ۱۳): آفلاین یا 5xx یعنی `Transient`؛ خطای اعتبارسنجی 4xx یعنی `Rejected`
- **اول ارسال، بعد دریافت:** پاسخ سرور همین حالا تغییرات پذیرفته‌شدهٔ شما را دارد
- تعداد تلاش را با `runAttemptCount` محدود کنید و به‌جای تلاش بی‌پایان، حالت «sync ممکن نیست» را نشان دهید

تلهٔ مصاحبه

برگرداندن `Result.failure()` روی خطای شبکه. کار برای همیشه تمام شده — تغییرات در انتظار تا یک محرک دیگر در outbox می‌مانند. مشکل گذرا یعنی `Result.retry()`.

تلهٔ مصاحبه

تلاش بی‌پایان برای عملیاتی که رد شده (4xx). این «پیام سمی» همهٔ عملیات پشت سرش را متوقف می‌کند. آن را `FAILED` کنید، به کاربر بگویید، و خالی کردن صف را ادامه دهید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. دریافت تغییرات: فقط تفاوت‌ها، نه دانلود کامل

```
suspend fun pullChanges() {
    var cursor = syncMetaDao.cursor()  // null = full sync
    do {
        val page = api.changes(since = cursor)
        db.withTransaction {
            noteDao.upsertIfNotPending(
                page.upserts.map { it.toEntity() },
            )
            noteDao.deleteByIds(page.deletedIds)
            syncMetaDao.setCursor(page.nextCursor)
        }
        cursor = page.nextCursor
    } while (page.hasMore)
}
```

- سرور فقط چیزهایی را برمی‌گرداند که بعد از یک **cursor** تغییر کرده‌اند (نسخهٔ change list یا یک توکن)
- داده **و** cursor جدید در یک تراکنش commit می‌شوند — هرگز crash تغییری را جا نمی‌اندازد
- پروژهٔ Now in Android هم همین ایده را دارد: change listهایی با نسخه برای هر نوع داده



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. چیزهایی که delta sync را خراب می‌کنند

تلهٔ مصاحبه

نگه داشتن cursor در DataStore در حالی که داده در Room است. دو ذخیره‌ساز، بدون تراکنش مشترک: crash بعد از ذخیرهٔ cursor و قبل از ردیف‌ها، آن تغییرات را **برای همیشه** جا می‌اندازد. cursor را در همان دیتابیس نگه دارید.

تلهٔ مصاحبه

فراموش کردن حذف‌ها. اگر سرور فقط ردیف‌های تغییرکرده را بفرستد، رکوردهایی که در سرور حذف شده‌اند روی همهٔ دستگاه‌ها زنده می‌مانند. فید تغییرات به **tombstone** (شناسه‌های حذف‌شده) نیاز دارد.

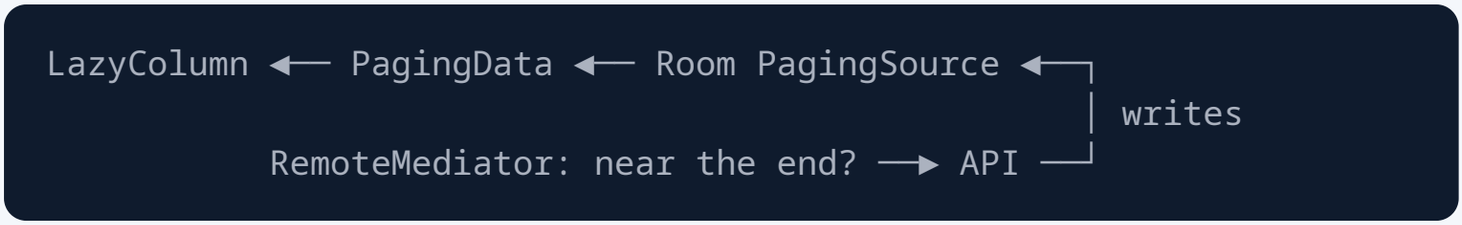
سؤال تکمیلی

«*اگر cursor خیلی قدیمی باشد چه؟*» — شاید سرور دیگر آن تاریخچه را نگه ندارد. باید جواب دهد «sync کامل لازم است»؛ کلاینت همه را دوباره دانلود می‌کند و نوشتن‌های محلیِ در انتظار را در outbox نگه می‌دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. صفحه‌بندی Paging 3 با RemoteMediator




```
@OptIn(ExperimentalPagingApi::class)
fun articles(): Flow<PagingData<Article>> = Pager(
    config = PagingConfig(pageSize = 30),
    remoteMediator = ArticleRemoteMediator(db, api),
    pagingSourceFactory = { db.articleDao().pagingSource() },
).flow.map { page -> page.map { it.toDomain() } }
```

- UI صفحه‌ها را **فقط از Room** می‌گیرد؛ وقتی لیست تمام می‌شود، mediator صفحه‌های شبکه را **در** Room می‌ریزد
- در حالت آفلاین، لیست هنوز در همهٔ داده‌های ذخیره‌شده اسکرول می‌شود
- در ViewModel از `.cachedIn(viewModelScope)` استفاده کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. داخل mediator

```
override suspend fun load(
    loadType: LoadType,
    state: PagingState<Int, ArticleEntity>,
): MediatorResult {
    val cursor = when (loadType) {
        LoadType.REFRESH -> null
        LoadType.PREPEND -> return MediatorResult.Success(true)
        LoadType.APPEND -> keys.nextCursor()
            ?: return MediatorResult.Success(true)
    }
    return try {
        val page = api.articles(cursor, state.config.pageSize)
        db.withTransaction {
            if (loadType == LoadType.REFRESH) dao.clearFeed()
            keys.save(page.nextCursor)
            dao.insertAll(page.items.map { it.toEntity() })
        }
        MediatorResult.Success(page.nextCursor == null)
    } catch (e: IOException) { MediatorResult.Error(e) }
}
```

تلهٔ مصاحبه

پاک کردن جدول در refresh **بیرون** از تراکنش — لیست یک لحظه خالی می‌شود. query صفحه‌بندی هم یک `ORDER BY` پایدار لازم دارد (مثلاً ستون موقعیت از سرور)؛ ردیف‌ها ترتیب درج تضمین‌شده ندارند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

تشخیص تعارض، حل تعارض، migration در مقیاس بزرگ



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۸. تشخیص تعارض

```
@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey val id: String,  // client-generated UUID
    val title: String,
    val body: String,
    val baseVersion: Long,       // server version we edited
    val syncState: SyncState,    // SYNCED, PENDING, FAILED
)
```

```
PUT /notes/42   If-Match: "7"
<- 200 OK  ETag: "8"          nobody else changed it
<- 412 Precondition Failed    server is at "9": conflict
```

هم‌روندی خوش‌بینانه: کلاینت نسخه‌ای را که از آن شروع کرده می‌فرستد؛ سرور فقط وقتی نوشتن را اعمال می‌کند که آن نسخه هنوز جاری باشد. خیلی از APIها به‌جای آن با `409 Conflict` جواب می‌دهند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. چرا نسخه، نه ساعت

- **سرور** نسخه را تعیین می‌کند (یک عدد صحیح یا ETag) — یک ترتیب برای همه
- مقدار `baseVersion` را از لحظهٔ شروع ویرایش نگه دارید، نه از آخرین دریافت
- دریافت تغییرات نباید روی ردیف `PENDING` بنویسد — این یعنی دور ریختن بی‌صدای ویرایش کاربر

تلهٔ مصاحبه

«*آخرین نوشتن برنده است، با مقایسهٔ timestamp*» (پیش‌فرض نسخهٔ پارسال). — ساعت دستگاه‌ها جابه‌جا می‌شود و کاربران عوضش می‌کنند؛ «آخرین» ویرایش ممکن است قدیمی‌ترین باشد. با نسخه‌هایی که سرور تعیین می‌کند مرتب کنید، یا با timestampی که خود سرور می‌زند.

تلهٔ مصاحبه

هیچ تشخیص تعارضی نداشتن. هر نوشتن یک بازنویسی کورکورانه است — تعارض باز هم رخ می‌دهد، فقط هیچ‌وقت نمی‌فهمید دادهٔ چه کسی از دست رفت.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. حل تعارض — برای هر نوع داده

استراتژی	نتیجه	مناسب برای
سرور برنده	ویرایش محلی دور ریخته (و نشان داده) می‌شود	دادهٔ محاسبه‌شده در سرور، قیمت، دسترسی‌ها
کلاینت برنده	روی سرور بازنویسی می‌کند	دادهٔ تک‌کاربره، تنظیمات دستگاه
آخرین نوشتن (به ترتیب سرور)	آخرین نوشتنِ پذیرفته‌شده می‌ماند	مقدارهای تکی: نام نمایشی، «آخرین بازدید»
ادغام سه‌طرفه در سطح فیلد	فیلدهای بدون هم‌پوشانی هر دو حفظ می‌شوند	پروفایل، فرم، متادیتای یادداشت
عملیات به‌جای state	«یک آیتم اضافه کن» در سرور اعمال می‌شود	شمارنده، سبد خرید، لایک
CRDT	خودکار همگرا می‌شود	ویرایش گروهی هم‌زمان
پرسیدن از کاربر	کاربر انتخاب یا ادغام می‌کند	سندهایی که هر دو ویرایش مهم‌اند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. ادغام سه‌طرفه

```
fun <T> merge3(base: T, local: T, remote: T): T = when {
    local == base -> remote      // only the server changed
    remote == base -> local      // only we changed
    else -> remote               // true conflict: policy
}

fun mergeNote(base: Note, local: Note, remote: Note) =
    remote.copy(
        title = merge3(base.title, local.title, remote.title),
        body = merge3(base.body, local.body, remote.body),
    )
```

base (آخرین نسخهٔ syncشده) را نگه دارید تا «من عوضش کردم» را از «آن‌ها عوضش کردند» تشخیص دهید.

تلهٔ مصاحبه

«برای پیام‌های چت از آخرین نوشتن برنده است استفاده کن» (مثال نسخهٔ پارسال). — پیام‌ها **فقط اضافه می‌شوند**: هر کدام شناسهٔ خودش را دارد که کلاینت ساخته، پس دو پیام هرگز تعارض ندارند. این استراتژی برای یک مقدار تکی است که هر دو طرف بازنویسی‌اش می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. migration در مقیاس بزرگ: نوشتنش

```
val MIGRATION_7_8 = object : Migration(7, 8) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            """CREATE TABLE orders_new (
                id TEXT NOT NULL PRIMARY KEY,
                total_cents INTEGER NOT NULL)""",
        )
        db.execSQL(
            """INSERT INTO orders_new (id, total_cents)
               SELECT id, CAST(ROUND(total * 100) AS INTEGER)
               FROM orders""",
        )
        db.execSQL("DROP TABLE orders")
        db.execSQL("ALTER TABLE orders_new RENAME TO orders")
    }
}
```

دستور کپی جدول روی همهٔ نسخه‌های SQLite کار می‌کند: ساختن جدول جدید، کپی با **یک دستور مجموعه‌ای** `INSERT … SELECT`، حذف جدول قدیمی و تغییر نام — و بعد ساختن دوبارهٔ ایندکس‌ها.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. migration در مقیاس بزرگ: تستش

```
@get:Rule
val helper = MigrationTestHelper(
    InstrumentationRegistry.getInstrumentation(),
    AppDatabase::class.java,
)

@Test fun migrate7To8_convertsMoney() {
    helper.createDatabase(DB, 7).apply {
        execSQL("INSERT INTO orders VALUES ('a', 12.5)")
        close()
    }
    val db = helper.runMigrationsAndValidate(
        DB, 8, true, MIGRATION_7_8,
    )
    db.query("SELECT total_cents FROM orders").use { c ->
        c.moveToFirst()
        assertEquals(1250L, c.getLong(0))
    }
}
```

تابع `runMigrationsAndValidate` نتیجه را با اسکیمای exportشده مقایسه می‌کند — همان چکی که Room روی دستگاه‌های واقعی انجام می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. قاعده‌ها برای میلیون‌ها دستگاه

- مقدار `exportSchema = true` و commit کردن JSON اسکیما (با پلاگین Gradle مربوط به Room و `room { schemaDirectory(…) }`) — مثال نسخهٔ پارسال آن را `false` گذاشته بود
- **کاربران نسخه‌ها را جا می‌اندازند:** Room مهاجرت‌ها را پشت هم اجرا می‌کند (از ۳ تا ۸، قدم به قدم)، پس هر قدم قدیمی در کد و در تست‌ها می‌ماند
- migration وقتی اجرا می‌شود که دیتابیس **اولین بار باز شود** — یک migration کند اولین صفحه را معطل می‌کند؛ SQL مجموعه‌ای بنویسید، نه حلقهٔ Kotlin روی ردیف‌ها
- از `AutoMigration` برای تغییرات ساده استفاده کنید؛ تغییر نام و حذف، یک spec لازم دارند، یعنی `@RenameColumn` و `@DeleteColumn`؛ تبدیل داده یک `Migration` دستی می‌خواهد

تلهٔ مصاحبه

fallback مخرب «محض احتیاط». برای یک cache خالص اشکالی ندارد؛ برای دادهٔ کاربر، با هر migration ناموفق پیش‌نویس‌ها و ورودی‌های outbox را بی‌صدا پاک می‌کند. به‌جایش migration را تست کنید.

تلهٔ مصاحبه

migrationی که اجرا می‌شود اما اسکیمای کمی متفاوت می‌سازد (یک `NOT NULL` جاافتاده، یک ایندکس فراموش‌شده). اعتبارسنجی Room هنگام باز شدن دیتابیس exception می‌دهد — در هر بار اجرای اپ. همین حلقهٔ crash دلیل اجباری بودن تست migration است.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

رمزنگاری داده روی دیسک، لایهٔ داده در KMP، sync برای پول



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. رمزنگاری داده روی دیسک — هر لایه چه می‌دهد

لایه	در برابر چه محافظت می‌کند	در برابر چه محافظت نمی‌کند
رمزنگاری دستگاه (file-based، استاندارد در دستگاه‌های امروزی)	گوشی گم‌شده یا دزدیده‌شده که قفل است	هر چیزی که وقتی قفل باز است اجرا شود
sandbox اپ (فایل‌ها و دیتابیس‌های خصوصی)	اپ‌های دیگر	روت، backupهایی که مستثنا نشده‌اند
رمزنگاری در سطح اپ با کلید نگه‌داری‌شده در Keystore (مثل SQLCipher و Tink)	فایل‌های کپی‌شده: استخراج، backup، dump جرم‌شناسی	کدی که روی دستگاهِ آلوده به‌جای اپ شما اجرا می‌شود
کلیدی که احراز هویت کاربر لازم دارد	استفاده از داده بدون حضور کاربر	هیچ چیز مجانی نیست: هزینهٔ UX، کلید ممکن است باطل شود (روز ۲۰)

کلیدهای Keystore **قابل خروج نیستند**، اما دستگاه آلوده هنوز می‌تواند از Keystore بخواهد از آن‌ها *استفاده* کند. رمزنگاری هزینهٔ حمله را بالا می‌برد؛ دستگاه را قابل اعتماد نمی‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. رمزنگاری Room، و تله‌ها

```
System.loadLibrary("sqlcipher")
val passphrase: ByteArray = keyManager.dbPassphrase()
val db = Room
    .databaseBuilder(ctx, AppDatabase::class.java, "app.db")
    .openHelperFactory(SupportOpenHelperFactory(passphrase))
    .build()
```

- passphrase: بایت‌های تصادفی که یک بار ساخته و **رمزشده با یک کلید Keystore** ذخیره می‌شوند — هرگز hard-code نشوند
- فایل‌ها و DataStore: بایت‌ها را با **Tink** رمز کنید (با keysetی که Keystore آن را رمز کرده). ذخیره‌سازی امن به‌صورت عمیق ← روز ۱۹

تلهٔ مصاحبه

«*DataStore با factoryهایش و Jetpack Security رمز می‌شود*» (نسخهٔ پارسال). — DataStore رمزنگاری داخلی ندارد، و کتابخانهٔ `security-crypto` (یعنی EncryptedFile و EncryptedSharedPreferences) **منسوخ (deprecated)** شده است. رمزنگاری را در یک serializer سفارشی با Tink انجام دهید.

تلهٔ مصاحبه

Auto Backup دیتابیس رمزشده را روی گوشی جدید برمی‌گرداند — اما کلیدهای Keystore منتقل نمی‌شوند. اپ نمی‌تواند بازش کند و crash می‌کند. آن را از backup مستثنا کنید، یا نبودن کلید را تشخیص دهید و دوباره sync کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. لایهٔ داده در KMP

```
// commonMain
@Database(entities = [ArticleEntity::class], version = 1)
@ConstructedBy(AppDatabaseConstructor::class)
abstract class AppDatabase : RoomDatabase() {
    abstract fun articleDao(): ArticleDao
}

// actual is generated by Room's KSP processor
expect object AppDatabaseConstructor :
    RoomDatabaseConstructor<AppDatabase> {
    override fun initialize(): AppDatabase
}

fun createDatabase(
    builder: RoomDatabase.Builder<AppDatabase>,
): AppDatabase = builder
    .setDriver(BundledSQLiteDriver())
    .setQueryCoroutineContext(Dispatchers.IO)
    .build()
```

هر پلتفرم فقط `Builder` را با مسیر فایل خودش می‌سازد؛ بقیه مشترک است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. چه چیزی مشترک، چه چیزی بومی

بخش	کجا زندگی می‌کند
repositoryها، mapperها، منطق sync و ادغام	`commonMain`
entityها، DAOها و migrationهای Room	`commonMain` (با Room KMP)
تنظیمات	`commonMain` (با DataStore KMP؛ مسیر فایل برای هر پلتفرم)
HTTP	`commonMain` (با Ktor)؛ engine برای هر پلتفرم: OkHttp یا Darwin
زمان‌بندی sync	بومی: WorkManager در اندروید، BGTaskScheduler در iOS
کلیدها و رمزها	interface در کد مشترک؛ پشتش Keystore یا Keychain

تلهٔ مصاحبه

تابع‌های DAO مسدودکننده (blocking) در کد مشترک. Room KMP لازم می‌داند تابع‌های DAO در `commonMain` یا `suspend` باشند یا یک نوع reactive مثل `Flow` برگردانند.

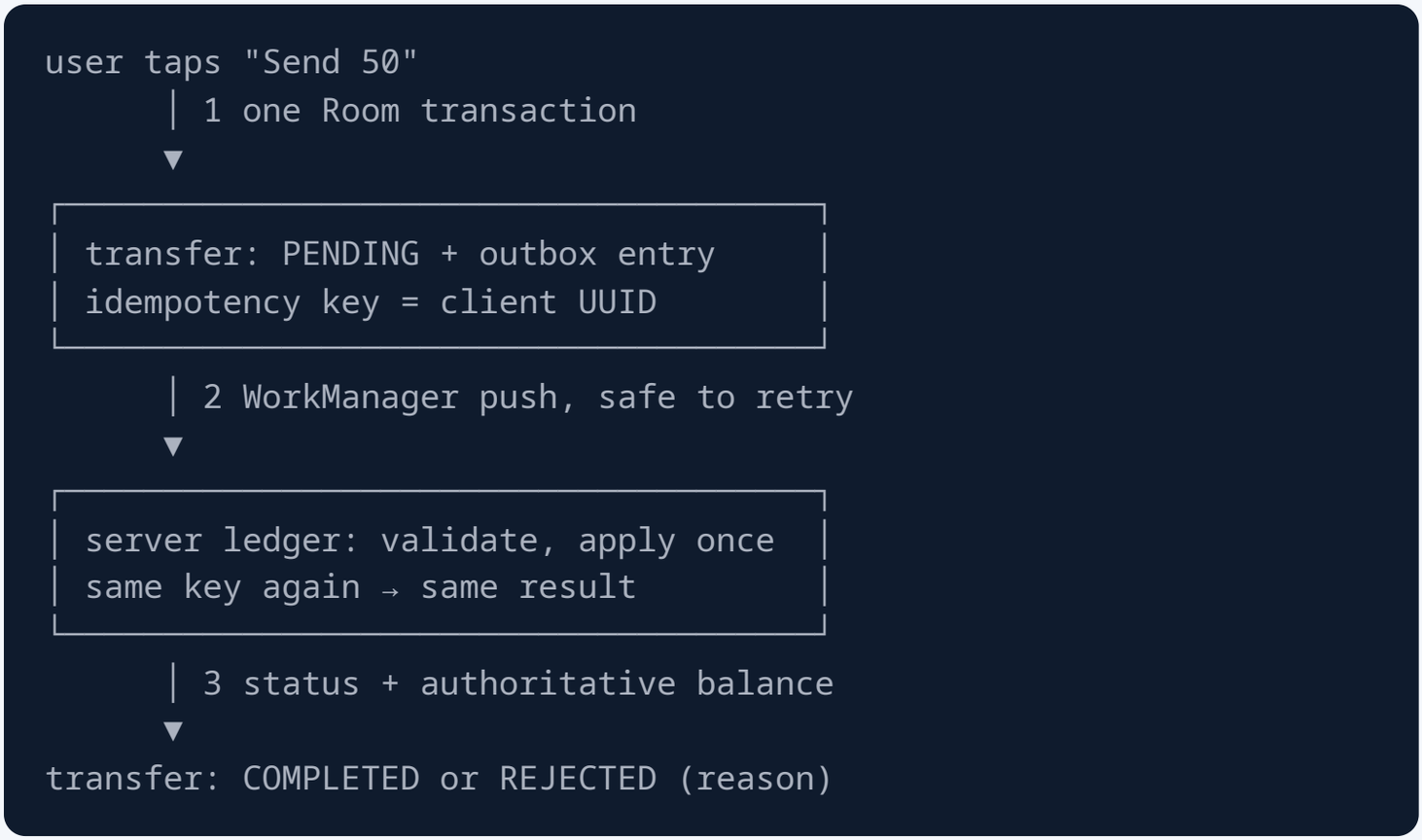
تلهٔ مصاحبه

«*KMP پس‌زمینهٔ sync را هم مشترک می‌کند.*» — **منطق** sync مشترک است؛ **زمان** اجرایش را هر سیستم‌عامل تعیین می‌کند. `Syncer` را مشترک کنید و به‌صورت بومی زمان‌بندی‌اش کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. sync برای پول: یک طراحی سرتاسری





اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. قاعده‌های پشت این نمودار

- کلاینت هرگز موجودی را ویرایش نمی‌کند؛ **فرمان** می‌فرستد. دفتر کل (ledger) سرور مرجع است
- آخرین موجودی تأییدشده را **همراه با زمانش** نشان دهید، و انتقال‌های در انتظار را جدا
- بعد از ارسال timeout شد؟ حدس نزنید — با **همان کلید** دوباره بفرستید، یا وضعیت همان کلید را از سرور بپرسید
- از نسخهٔ محلی محافظت کنید: دیتابیس رمزشده، پاک کردن در خروج از حساب، هیچ مبلغ یا توکنی در log
- خط لوله را پایش کنید: عمق outbox، سن قدیمی‌ترین عملیات در انتظار، دلیل‌های رد شدن

تلهٔ مصاحبه

«*بگذار کاربر تراکنش‌های متعارض را حل کند*» (نسخهٔ پارسال). — پول ادغام نمی‌شود. دو انتقال که حساب را منفی می‌کنند تعارضی برای انتخاب نیستند: سرور یکی را با دلیل **رد می‌کند** و اپ آن را نشان می‌دهد.

جواب سینیور در یک جمله

«از دیتابیس محلی بخوان، محلی و همراه با outbox بنویس، با فرمان‌های idempotent در WorkManager همگام کن، تعارض را با نسخهٔ سرور تشخیص بده، برای هر نوع داده جدا حلش کن — و هر migration را تست کن.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۴ — خلاصهٔ یک‌صفحه‌ای

- در offline-first، UI فقط از دیتابیس محلی می‌خواند؛ **نوشتن‌ها** آفلاین پذیرفته و بعداً sync می‌شوند.
- وضعیت sync داده است و در ردیف ذخیره می‌شود. شناسه را کلاینت می‌سازد.
- ردیف و ورودی outbox در **یک تراکنش**؛ هر عملیات یک idempotency key دارد.
- WorkManager: کار یکتا، `CONNECTED` برای نوشتن کاربر، `retry()` برای خطای گذرا، `FAILED` برای 4xx.
- تفاوت‌ها را با cursorی بگیرید که **در همان دیتابیس** است؛ tombstone برای حذف‌ها.
- در Paging 3، UI از Room صفحه می‌گیرد و `RemoteMediator` آن را پر می‌کند.
- تعارض را با نسخهٔ سرور (`If-Match`) تشخیص دهید، نه ساعت دستگاه؛ برای هر نوع داده جدا حلش کنید.
- migration: اسکیمای exportشده، تست همهٔ مسیرها، SQL مجموعه‌ای، بدون fallback مخرب برای دادهٔ کاربر.
- رمزنگاری: کلید پیچیده در Keystore، SQLCipher یا Tink؛ `security-crypto` منسوخ است؛ حواستان به backup باشد.
- KMP: کد Room، DataStore، Ktor و منطق sync مشترک؛ زمان‌بندی بومی.

روز ۱۵ در راه است: کار پس‌زمینه بخش ۱ — WorkManager، حالت Doze و محدودیت‌های پس‌زمینه. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700